

PIXOUS TECHNOLOGIES — ESTIMATION & INVOICING API

# Unit Test Documentation

---

Module-Level Coverage: Security Utilities, Invoice Builder, Auth  
Helpers

---

**Generated:** August 14, 2026

**Source:** UNIT\_TESTING.md

**Test framework:** Pytest + FastAPI TestClient

# Contents

---

## [1. Framework & conventions](#)

## [2. How to run](#)

## [3. tests/test\\_unit\\_security\\_utils.py](#)

[3.1 utils/security.py — password hashing](#)

[3.2 utils/rate\\_limiter.py — login lockout](#)

[3.3 utils/key\\_manager.py — API key round-robin/failover](#)

## [4. tests/test\\_unit\\_invoice\\_builder.py](#)

## [5. tests/test\\_unit\\_api\\_auth\\_helpers.py](#)

Scope: `tests/test_unit_security_utils.py`, `tests/test_unit_invoice_builder.py`, `tests/test_unit_api_auth_helpers.py`.

This document covers **unit tests only** — tests that exercise a single module's pure/deterministic logic directly (no HTTP layer, no end-to-end flow). For integration/E2E coverage over the FastAPI app, see the main [TESTING.pdf](#). For authentication/authorization/injection-focused tests, see [SECURITY\\_TESTING.pdf](#).

## 1. Framework & conventions

- **Framework:** Pytest.
- **Pattern:** Arrange–Act–Assert. Test names state the condition and the expected behavior:  
`test_<subject>_<condition>_<expected_outcome>`.
- **Mocking policy:** The system under test is never mocked. The only thing ever patched in this layer is `time.time()` (via `monkeypatch`) to make sliding-window/lockout-expiry logic in the rate limiter deterministic without real sleeps. `bcrypt`, `KeyManager`'s `asyncio.Lock`, and `invoice_builder`'s HTML generation all run for real.
- **No DB, no network:** none of these tests touch the database or the network. `test_unit_api_auth_helpers.py` uses the `db_session` fixture from `conftest.py` only for the two `is_valid_username` cases that need a real `User` row to query against — that fixture still points at the isolated temp SQLite file, never production data.

## 2. How to run

```
# All unit tests
python -m pytest tests/test_unit_security_utils.py tests/test_unit_invoice_builder.py
tests/test_unit_api_auth_helpers.py -v

# One file
python -m pytest tests/test_unit_invoice_builder.py -v

# One test
python -m pytest
tests/test_unit_security_utils.py::test_hash_password_is_salted_so_same_input_yields_different_hashes -v
```

Current status: **42 passed**, 0 failed.

## 3. `tests/test_unit_security_utils.py`

Modules under test: `utils/security.py`, `utils/rate_limiter.py`, `utils/key_manager.py`.

### 3.1 `utils/security.py` — password hashing

| Test                                                                            | Scenario                                            | Expected behavior                                                    |
|---------------------------------------------------------------------------------|-----------------------------------------------------|----------------------------------------------------------------------|
| <code>test_hash_password_returns_a_bcrypt_hash_distinct_from_input</code>       | Hash a plaintext password                           | Result is a recognizable bcrypt hash and is never equal to the input |
| <code>test_hash_password_is_salted_so_same_input_yields_different_hashes</code> | Hash the same password twice                        | The two hashes differ (bcrypt salt is randomized per call)           |
| <code>test_verify_password_accepts_correct_bcrypt_hash</code>                   | Verify the correct password against its bcrypt hash | Returns <code>True</code>                                            |
| <code>test_verify_password_rejects_wrong_password_against_bcrypt_hash</code>    | Verify an incorrect password against a bcrypt hash  | Returns <code>False</code>                                           |

| Test                                                                                     | Scenario                                                                                                                          | Expected behavior                                                                          |
|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <code>test_verify_password_accepts_matching_legacy_plaintext</code>                      | Verify against a legacy (pre-bcrypt) plaintext row where the password matches                                                     | Returns <code>True</code>                                                                  |
| <code>test_verify_password_rejects_mismatched_legacy_plaintext</code>                    | Verify against a legacy plaintext row where the password does not match                                                           | Returns <code>False</code>                                                                 |
| <code>test_verify_password_rejects_when_stored_is_empty_or_none</code>                   | Stored hash is <code>""</code> or <code>None</code>                                                                               | Returns <code>False</code> for both — never raises                                         |
| <code>test_verify_password_handles_malformed_bcrypt_prefixed_hash_without_raising</code> | Stored value starts with a bcrypt prefix ( <code>\$2b\$</code> ) but isn't a well-formed hash (e.g. corrupted by a bad migration) | Returns <code>False</code> — fails closed instead of raising and 500ing the login endpoint |
| <code>test_is_bcrypt_hash_false_for_none_and_plain_strings</code>                        | <code>is_bcrypt_hash(None)</code> ,<br><code>is_bcrypt_hash("")</code> ,<br><code>is_bcrypt_hash("plaintext-password")</code>     | All return <code>False</code>                                                              |
| <code>test_is_bcrypt_hash_true_for_all_supported_prefixes</code>                         | A well-formed hash string for each of <code>\$2a\$</code> / <code>\$2b\$</code> / <code>\$2y\$</code>                             | All return <code>True</code>                                                               |

**Why these edge cases matter:** the malformed-hash and empty/ `None` -stored cases are the two failure modes that would otherwise turn a login attempt into an unhandled exception (a 500) instead of a clean authentication failure (a 401) — the boundary between "wrong password" and "broken data" must fail safely either way.

### 3.2 `utils/rate_limiter.py` — login lockout

An `autouse` fixture ( `_clear_rate_limiter_state` ) clears the module's process-global `_failures` / `_locked_until` dicts before and after every test in this section, since the rate limiter intentionally has no other reset mechanism (it's designed to persist for the life of the process).

| Test                                                                                                    | Scenario                                                                       | Expected behavior                                                                                                                 |
|---------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <code>test_check_login_rate_limit_returns_none_when_no_failures_recorded</code>                         | Fresh (ip, username) pair                                                      | <code>check_login_rate_limit</code> returns <code>None</code> (not locked out)                                                    |
| <code>test_record_login_failure_below_threshold_does_not_lock_out</code>                                | <code>MAX_FAILED_ATTEMPTS - 1</code> failures recorded                         | Still not locked out                                                                                                              |
| <code>test_record_login_failure_at_threshold_locks_out_with_correct_duration</code>                     | <code>MAX_FAILED_ATTEMPTS</code> failures recorded                             | Locked out; remaining seconds is <code>&gt; 0</code> and <code>&lt;= LOCKOUT_SECONDS</code>                                       |
| <code>test_lockout_key_is_scoped_to_ip_and_username_case_insensitively</code>                           | Lock out (ip, "Alice") , then check (ip, "alice") and (other_ip, "alice")      | Same-ip/different-case is still locked out (case-insensitive username key); different-ip is not locked out (scoped per source IP) |
| <code>test_record_login_success_clears_failures_and_lockout</code>                                      | Lock out a pair, then record a success for it                                  | Lockout is cleared immediately                                                                                                    |
| <code>test_lockout_expires_after_lockout_window_elapses</code> (monkeypatch <code>time.time</code> )    | Lock out at <code>t=0</code> , check at <code>t=LOCKOUT_SECONDS+1</code>       | No longer locked out                                                                                                              |
| <code>test_failures_outside_sliding_window_are_not_counted</code> (monkeypatch <code>time.time</code> ) | Record <code>MAX_FAILED_ATTEMPTS - 1</code> failures at <code>t=0</code> , one | Old failures have aged out of the sliding window, so the pair is still below threshold and not locked out                         |

| Test | Scenario                                   | Expected behavior |
|------|--------------------------------------------|-------------------|
|      | more at<br><code>t=WINDOW_SECONDS+1</code> |                   |

**Why these edge cases matter:** the case-insensitivity and per-(ip, username) scoping tests exist because a rate limiter scoped incorrectly either fails to protect an account (case mismatch bypass) or accidentally locks out unrelated legitimate users sharing an IP (see the companion integration test in [SECURITY\\_TESTING.pdf § test\\_rate\\_limit\\_is\\_scoped\\_per\\_username\\_not\\_globally](#)). The window/expiry tests exist because a sliding window implemented with an unbounded list, or a lockout with no expiry, are both easy off-by-one mistakes that only surface under time pressure.

### 3.3 `utils/key_manager.py` — API key round-robin/failover

All async tests wrap their body in a local `_run()` coroutine and drive it with `asyncio.run(...)`, since `KeyManager` uses `asyncio.Lock` internally.

| Test                                                                        | Scenario                                                              | Expected behavior                                                                                                                                |
|-----------------------------------------------------------------------------|-----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>test_key_manager_requires_at_least_one_key</code>                     | Construct with <code>[]</code>                                        | Raises <code>ValueError</code>                                                                                                                   |
| <code>test_key_manager_round_robins_across_keys</code>                      | 3 keys, call <code>get_key()</code> 6 times                           | Returns <code>[k1, k2, k3, k1, k2, k3]</code> in strict rotation                                                                                 |
| <code>test_key_manager_skips_keys_marked_failed</code>                      | Mark <code>k2</code> failed, then call <code>get_key()</code> 4 times | <code>k2</code> never returned; only <code>k1 / k3</code> cycle                                                                                  |
| <code>test_key_manager_resets_and_recovers_when_all_keys_have_failed</code> | Mark every key failed, then call <code>get_key()</code>               | Does not deadlock or raise — resets the failed set and returns a key; <code>available_count</code> returns to <code>total_count</code> afterward |
| <code>test_key_manager_available_and_total_count_reflect_failures</code>    | Mark 1 of 3 keys failed                                               | <code>total_count == 3</code> ,<br><code>available_count == 2</code>                                                                             |
| <code>test_key_manager_reset_clears_all_failed_keys</code>                  | Mark a key failed, call <code>reset()</code>                          | <code>available_count == total_count</code> again                                                                                                |

**Why these edge cases matter:** the all-keys-failed case is the one that would silently deadlock a production LLM-key rotator (every remaining provider request blocking forever) if the reset-on-exhaustion branch were missing or buggy — this is the single most important case in the file.

## 4. `tests/test_unit_invoice_builder.py`

**Module under test:** `invoice_builder.build_invoice` — pure, deterministic HTML generation from estimation data (no LLM, no DB, no network call of any kind).

| Test                                                                             | Scenario                                                              | Expected behavior                                                                                                         |
|----------------------------------------------------------------------------------|-----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <code>test_build_invoice_computes_tax_and_total_from_subtotal</code>             | <code>grand_total=780000</code> ,<br><code>tax_percentage=18.0</code> | <code>meta["subtotal"] == 780000</code> , <code>tax_amount ≈ 780000*0.18</code> ,<br><code>total_due ≈ 780000*1.18</code> |
| <code>test_build_invoice_due_date_offsets_from_invoice_date_by_due_days</code>   | <code>due_days=30</code> , explicit<br><code>invoice_date</code>      | <code>due_date - invoice_date == timedelta(days=30)</code> exactly                                                        |
| <code>test_build_invoice_includes_bank_box_only_when_bank_name_configured</code> | Build once with <code>bank_name</code> set, once without              | Bank box markup present only in the first case                                                                            |

| Test                                                                                | Scenario                                                                                                                        | Expected behavior                                                                                                                                                                                                 |
|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>test_build_invoice_line_items_sorted_by_cost_descending</code>                | Two role estimates, one with a higher <code>total_cost</code>                                                                   | Higher-cost line item appears earlier in the rendered HTML                                                                                                                                                        |
| <code>test_build_invoice_adds_extra_rows_for_infra_license_and_contingency</code>   | Non-zero <code>infrastructure_cost_monthly</code> , <code>third_party_licenses_monthly</code> , <code>contingency_amount</code> | All three corresponding extra line-item rows appear in the HTML                                                                                                                                                   |
| <code>test_build_invoice_escapes_malicious_client_name_to_prevent_xss</code>        | <code>client_name = "&lt;script&gt;alert(1)&lt;/script&gt;"</code>                                                              | Raw <code>&lt;script&gt;</code> tag never appears in the rendered HTML ( <code>&amp;lt;script&amp;gt;</code> does); the <i>unescaped</i> raw value is still returned in <code>meta</code> for callers who need it |
| <code>test_build_invoice_escapes_malicious_bank_details</code>                      | <code>bank_name = '"&gt;&lt;img src=x onerror=alert(1)&gt;'</code>                                                              | No real <code>&lt;img&gt;</code> tag is formed in the output ( <code>&lt;img src=x"</code> not in html ); the escaped literal text is present instead                                                             |
| <code>test_build_invoice_falls_back_to_unspecified_client_when_no_name_given</code> | No <code>client_name</code> anywhere in the input data                                                                          | <code>meta["client_name"] == "Unspecified Client"</code> , and that string appears in the HTML                                                                                                                    |
| <code>test_build_invoice_handles_no_line_items_without_raising</code>               | <code>role_estimates = []</code> , <code>grand_total = 0</code>                                                                 | Does not raise; <code>subtotal / total_due</code> are <code>0</code> ; an (empty) <code>&lt;tbody&gt;</code> is still emitted                                                                                     |
| <code>test_build_invoice_uses_supplied_status_for_badge_and_meta</code>             | <code>status="Paid"</code>                                                                                                      | <code>meta["status"] == "Paid"</code> ; the "Paid" status badge color ( <code>#DCFCE7</code> ) appears in the HTML                                                                                                |

**Design note — asserting structural safety, not string absence:** for attacker-controlled strings rendered into HTML (the two XSS tests above), the correct assertion is "no real tag/attribute is formed" ( `"<img src=x" not in html` ), not "the substring `onerror=alert(1)` never appears anywhere." `html.escape()` neutralizes the angle brackets and quotes that actually matter for breaking out of a text node; the escaped text naturally still *contains* those words as harmless text. Asserting plain substring absence here would be a false requirement that doesn't correspond to any real vulnerability.

## 5. `tests/test_unit_api_auth_helpers.py`

**Module under test:** `api.py`'s token/identity primitives — `decode_token`, `verify_token`, `is_valid_username` — tested directly and independently of the HTTP/middleware layer that calls them.

| Test                                                                                | Scenario                                                                          | Expected behavior                                                  |
|-------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|--------------------------------------------------------------------|
| <code>test_decode_token_returns_payload_for_a_validly_signed_unexpired_token</code> | A token forged the same way <code>api.create_token</code> builds one              | Returns the decoded <code>{"user": ..., "exp": ...}</code> payload |
| <code>test_decode_token_returns_none_for_tampered_signature</code>                  | Last 4 characters of a valid token's signature replaced                           | Returns <code>None</code>                                          |
| <code>test_decode_token_returns_none_for_expired_token</code>                       | Token with <code>exp</code> in the past                                           | Returns <code>None</code>                                          |
| <code>test_decode_token_returns_none_for_malformed_token_shape</code>               | Not base64/JSON at all, empty string, or too many <code>.</code> -separated parts | Returns <code>None</code> for all three — never raises             |

| Test                                                                               | Scenario                                                                               | Expected behavior                                                                                                                                   |
|------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>test_verify_token_is_a_thin_boolean_wrapper_over_decode_token</code>         | Valid token vs. the same token with a tampered signature                               | <code>True</code> then <code>False</code> — confirms <code>verify_token</code> is exactly <code>decode_token(token)</code> is not <code>None</code> |
| <code>test_is_valid_username_true_for_existing_user_row</code>                     | A real <code>User</code> row exists for the username                                   | Returns <code>True</code>                                                                                                                           |
| <code>test_is_valid_username_true_for_bootstrap_admin_even_without_user_row</code> | Username equals <code>config.ADMIN_USERNAME</code> , no matching <code>User</code> row | Returns <code>True</code> (the same bootstrap carve-out <code>login_endpoint</code> trusts)                                                         |
| <code>test_is_valid_username_false_for_unknown_username</code>                     | Username matches neither a <code>User</code> row nor <code>ADMIN_USERNAME</code>       | Returns <code>False</code>                                                                                                                          |
| <code>test_is_valid_username_false_for_empty_or_none</code>                        | <code>""</code> and <code>None</code>                                                  | Both return <code>False</code>                                                                                                                      |

**Why this file exists separately from the security suite:** these are the exact primitives `auth_middleware` and `get_current_username` rely on to close the JWT-forgery gap described in `SECURITY_TESTING.pdf` §2. Unit tests here isolate their logic from the HTTP layer so a future change to the middleware's plumbing can't accidentally leave a hole in `decode_token` / `is_valid_username` unnoticed just because the higher-level HTTP test happened to still pass for an unrelated reason.